

Java Generics

Page No.:

Date:

YOUVA
STELLAR

(Type Safety)

Set of Rules

10 → Integer
"Hello" → String
true → Boolean
values → data types

Integer → Arithmetic operⁿ
String → String Methods

Types → Rules

Java → Typed language

* Upcasting & Downcasting

~~int~~ long l = 123456789;
int i = l;

int i = (int) l;

* Upcasting

```
class Animal {  
}
```

```
class Dog extends Animal {  
}
```

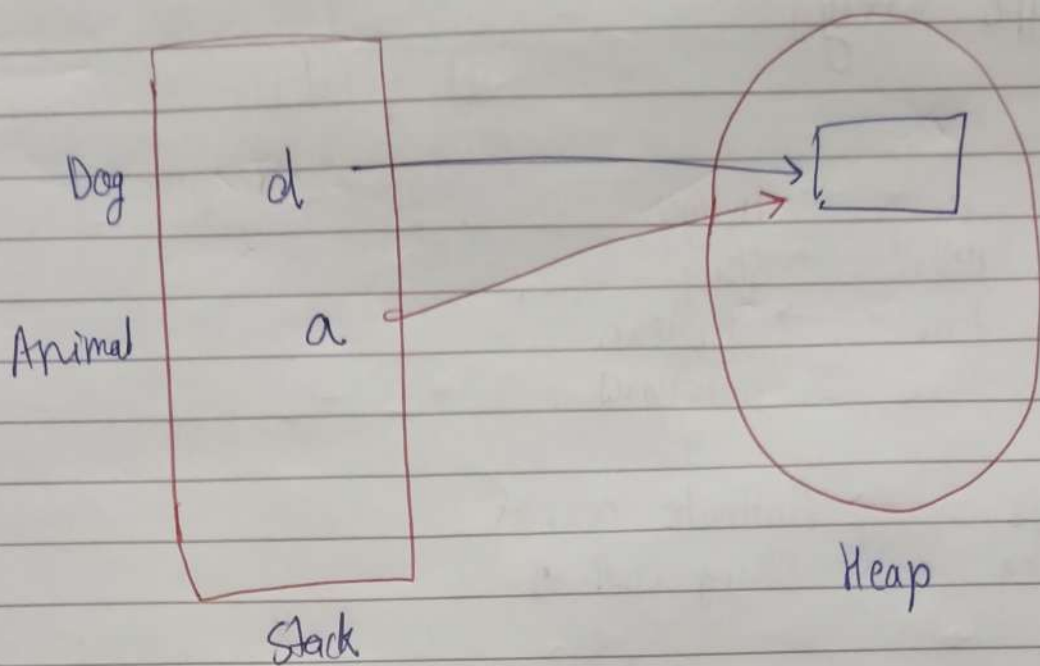
Animal a = new Dog();
Dog d = new Dog();

Animal {
 walk()
 run()
 eat()
}

Dog {
 bark()
}

a = d

a.bark() ✗
d.bark() ✓

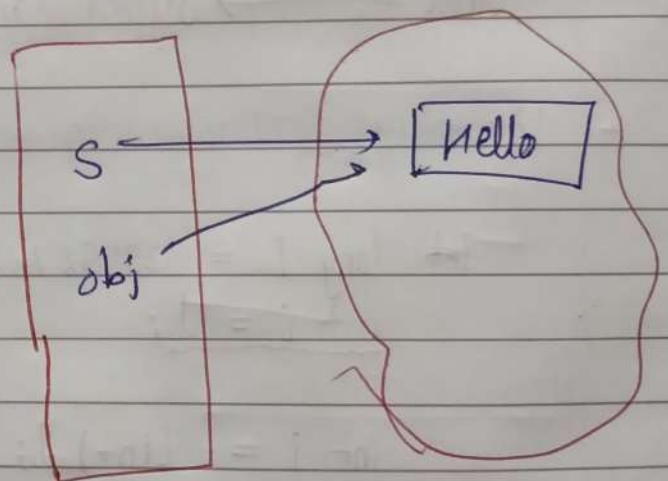


Specific class $\longrightarrow$ General class

```
String s = "Hello";
Object obj = s;
```

No casting keyword (X)

```
(int x;
 long l = x;
```

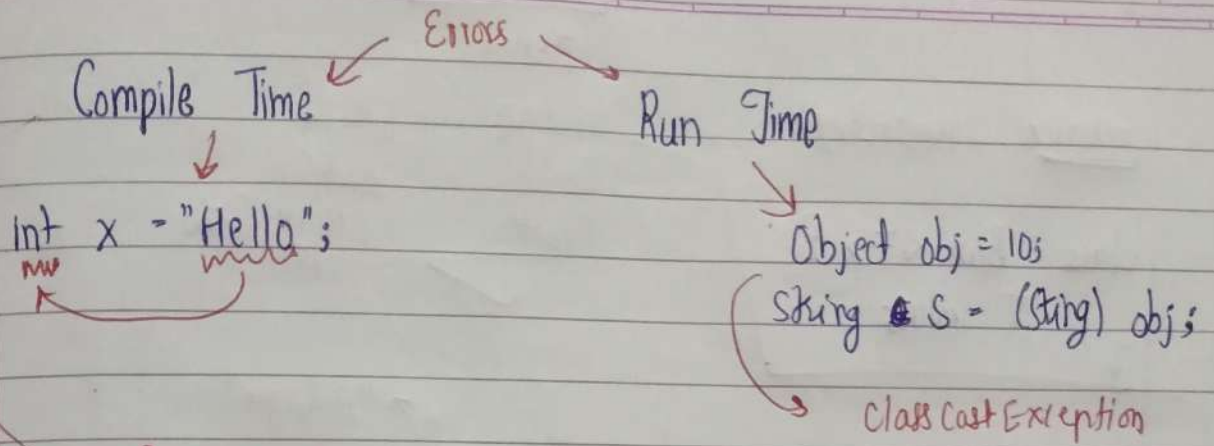


* Down casting;

```
Object obj = "Hello";
String s = obj; // compiler error
String s = (String) obj;
```

```
Object obj = 10; // int
String s = (String) obj;
```

ClassCastException



- Production Bug
- Code is safe
- Better IDE Support
- less Debugging

* ~~Generics~~

* limitations of using Object as Universal Type

- Type information is lost
- Wrong ~~obj~~ object could be inserted.
- Casting become necessary when reading
- Many errors shift to Runtime

↓ ↓ so we ~~are~~ needed

generics

Number

↑ extend

Generics (Restricting)

class Box <T> {

T values

}

Integer
Double
FloatString
Boolean
String X

→ "N-ditya"

Box<String> b1 = new Box<>(~~"N-ditya"~~);